Trees, Traversals & Binary Search Trees

COMS 2132 — Lab 6

Today's Agenda

Trees & Traversals

- Binary Tree Basics (refresher)
- Preorder, Inorder, Postorder
- Breadth-First (Level Order)

BSTs & Practice

- Binary Search Tree Property
- BST Find & Insert
- LeetCode Examples

PART 1

Trees & Traversals

Structure, terminology & traversal algorithms

What is a Tree?

A tree is an abstract model of a hierarchical structure — one of the most ubiquitous data structures in computer science.

```
        A          ← root
      /   \
    B       C      ← internal
   / \       \
  D   E       F    ← leaves
     / \
    G   H          ← leaves
```

Key Terminology

Root Top node with no parent

Internal Node with at least one child

Leaf Node with no children (external)

Depth Number of ancestors of a node

Height Maximum depth of any node

Siblings Children of the same parent

Binary Trees — Structure

A binary tree is an ordered tree where every node has at most two children (left and right).

```
class Node:
  def __init__(self, element,
               left=None, right=None):
    self.element = element
    self.left = left
    self.right = right

class BinaryTree:
  def __init__(self):
    self.root = None
```

Properties of Binary Trees

- Each level d has at most 2^d nodes
- Height h tree: at least h+1 nodes, at most $2^{h+1}-1$ nodes
- A full binary tree: every node has 0 or 2 children
- A complete tree: all levels full except possibly last

💡 $\log_2(n+1) - 1 \leq h \leq n - 1$

Computing Height — Recursion

The height of a tree is computed recursively: base case is an empty subtree (height = −1), and each node's height is 1 + max(left height, right height).

```
def _height(self, node):
    if node is None:
        return -1
    left_h = self._height(node.left)
    right_h = self._height(node.right)
    return max(left_h, right_h) + 1

def height(self):
    return self._height(self.root)
```

Walkthrough

```
        8          h=2
       / \
      4   10       h=1
     /
    2              h=0
```

```
height(10) = max(−1, −1) + 1 = 0
height(2) = max(−1, −1) + 1 = 0
height(4) = max(0, −1) + 1 = 1
height(8) = max(1, 0) + 1 = 2
```

Runtime: O(n) — every node visited exactly once. Space: O(h) for the recursion stack.

Tree Traversals — Overview

Four standard ways to visit every node in a binary tree:

Preorder (DFS)

Visit root, then left, then right

`Root → Left → Right`

Inorder (DFS)

Visit left, then root, then right

`Left → Root → Right`

Postorder (DFS)

Visit left, then right, then root

`Left → Right → Root`

Level-Order (BFS)

Visit level by level, left to right

`Uses a Queue`

DFS Traversals — Code

```
      4
     / \
    2   6
   / \ / \
  1  3 5  7
```

Preorder

```
def preorder(node):
  if node is None:
    return
  visit(node)      # root
  preorder(node.left)
  preorder(node.right)
```

→ 4, 2, 1, 3, 6, 5, 7

Inorder

```
def inorder(node):
  if node is None:
    return
  inorder(node.left)
  visit(node)      # root
  inorder(node.right)
```

→ 1, 2, 3, 4, 5, 6, 7

Postorder

```
def postorder(node):
  if node is None:
    return
  postorder(node.left)
  postorder(node.right)
  visit(node)      # root
```

→ 1, 3, 2, 5, 7, 6, 4

All DFS traversals are O(n) time, O(h) space. Inorder on a BST yields sorted order!

Breadth-First (Level-Order) Traversal

Visit all nodes level by level, left to right. Uses a queue instead of recursion.

```
def breadth_first(root):
    if root is None:
        return
    queue = [root]
    while queue:
        node = queue.pop(0)
        visit(node)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
```

Step-by-Step

```
        4
       / \
      2   6
     / \ / \
    1  3 5  7
```

```
Queue: [4] → visit 4
Queue: [2, 6] → visit 2
Queue: [6, 1, 3] → visit 6
Queue: [1, 3, 5, 7] → ...
Output: 4, 2, 6, 1, 3, 5, 7
```

💡 Time: O(n) — visit every node once. Space: O(w) where w = max width of the tree (up to n/2 at last level).

PART 2

Binary Search Trees

The BST property, find, insert & performance

The BST Property

A Binary Search Tree is a binary tree where for every node:

All keys in left subtree < node's key

All keys in right subtree > node's key

Both subtrees are also BSTs

Valid BST

```
        8
       / \
      4   12
     / \   / \
    2   6 10  14
```

Inorder: 2,4,6,8,10,12,14 ✓ sorted

NOT a Valid BST

```
      5
     / \
    1   4        ← 4 < 5!
       / \
      3   6
```

3 is in right subtree but 3 < 5 ✗

BST — Find & Insert

find(element) — O(h)

```
def _find(self, node, element):
    if node is None:
        return None
    if element < node.element:
        return self._find(
            node.left, element)
    elif element > node.element:
        return self._find(
            node.right, element)
    else:
        return node  # found!
```

Go left if smaller, right if larger, stop when found.

insert(element) — O(h)

```
def _insert(self, node, element):
    if node is None:
        return Node(element,
                    None, None)
    if element < node.element:
        node.left = self._insert(
            node.left, element)
    elif element > node.element:
        node.right = self._insert(
            node.right, element)
    return node
```

Navigate to None spot, create new node, link parent.

⚠ h = O(log n) for a balanced tree, but h = O(n) for a degenerate (linked-list) tree! Sorted input ⟶ worst case.

BST Performance — Balanced vs Degenerate

Balanced — O(log n)
left and right subtrees of every node differ in height by at most 1

```
        8
       / \
      4   12
     / \   / \
    2   6 10  14
   /
  1
```

8 nodes, height = 3
find/insert = O(log n)

Degenerate — O(n)

```
1
 \
  2
   \
    4
     \
      6  ... (sorted insert!)
```

8 nodes, height = 7
find/insert = O(n)

Operation	Balanced BST	Degenerate BST	Unsorted Array
Find	O(log n)	O(n)	O(n)
Insert	O(log n)	O(n)	O(1) amortized
Inorder traversal	O(n) — sorted!	O(n)	O(n log n) sort

PART 3

Practice

Applying tree & BST concepts to examples

Example #1: Inverted Binary Tree

Given the root of a binary tree, invert it — swap every node's left and right children — and return the root.

Input

```
      4
     / \
    2   7
   / \ / \
  1  3 6  9
```

Output (Inverted)

```
      4
     / \
    7   2
   / \ / \
  9  6 3  1
```

Key Insight

Every subtree is itself a smaller binary tree.
Recursively swap left and right at each node.

Approach 1: DFS

Recursive swap

Approach 2: BFS

Queue-based swap

Example #1: DFS Solution

Optimal

Swap left and right children at each node, then recurse into both subtrees.

```
def invertTree(root):
    if root is None:
        return None

    # Swap children
    root.left, root.right = root.right, root.left

    # Recurse
    invertTree(root.left)
    invertTree(root.right)

    return root
```

Recursive Walkthrough

```
invertTree(4):
  swap 2 ↔ 7  →  (7, 2)
  invertTree(7):
    swap 6 ↔ 9  →  (9, 6)
    invertTree(9): leaf ✓
    invertTree(6): leaf ✓
  invertTree(2):
    swap 1 ↔ 3  →  (3, 1)
    invertTree(3): leaf ✓
    invertTree(1): leaf ✓
```

Time: O(n) — visit every node once. Space: O(h) — recursion stack depth (O(log n) balanced, O(n) worst).

Example #2: Validate BST

Given the root of a binary tree, determine if it is a valid BST.

Output: True

```
    2
   / \
  1   3
```

`1 < 2 < 3` ✓

Output: False

```
      5
     / \
    1   4
       / \
      3   6
```

`4 is in right subtree but 4 < 5` ✗

Key Insight

Don't just check node vs. its children — a node must satisfy bounds from all ancestors.
Each node has a valid range: (min_val, max_val) that narrows as you descend.

Approach 1: Inorder traversal — check sorted

Approach 2: Recursive bounds checking

Example #2: Inorder Approach

A BST's inorder traversal produces a strictly increasing sequence. Just do an inorder traversal and check.

```
def isValidBST(root):
    prev = [None]  # track previous value

    def inorder(node):
        if node is None:
            return True
        if not inorder(node.left):
            return False
        if (prev[0] is not None and
                node.val <= prev[0]):
            return False
        prev[0] = node.val
        return inorder(node.right)

    return inorder(root)
```

How It Works

```
Valid BST [2,1,3]:
  Inorder: 1, 2, 3
  1 < 2 < 3  → True ✓

Invalid [5,1,4,null,null,3,6]:
  Inorder: 1, 5, 3, 4, 6
  5 > 3  → False ✗

If any element ≤ previous,
it's not a valid BST.
```

Time: O(n) — full inorder traversal. Space: O(h) — recursion stack. Early exit on first violation.

Example #2: Bounds Approach

Pass down valid (min, max) bounds as you recurse. Each node must fall strictly within its bounds.

```
def isValidBST(root):
    def validate(node, lo, hi):
        if node is None:
            return True
        if node.val <= lo or node.val >= hi:
            return False
        return (
          validate(node.left, lo, node.val) and
          validate(node.right, node.val, hi)
        )
    return validate(root, -inf, inf)
```

Bounds Walkthrough

```
Tree: [5, 1, 4, null, null, 3, 6]

validate(5, -∞, ∞)  → ok
  validate(1, -∞, 5) → ok
  validate(4, 5, ∞)
    4 <= 5?  → False! ✗

  4 must be > 5 (right of 5)
  but 4 < 5 → invalid
```

	Inorder Approach	Bounds Approach
Strategy	Check sorted order	Pass (lo, hi) range
Early exit?	Yes, on first violation	Yes, on first violation
Intuition	BST ↔ sorted inorder	**Each node bounded by ancestors**

Example #3: Maximum Depth of Binary Tree

Given the root of a binary tree, return its maximum depth (# of nodes along longest root-to-leaf path).

Example

```
        3          depth 0
       / \
      9  20        depth 1
        / \
       15   7      depth 2
```

Output: 2

Recursive DFS

```
def maxDepth(root):
    if root is None:
        return -1
    left = maxDepth(root.left)
    right = maxDepth(root.right)
    return max(left, right) + 1
```

Example #3: Maximum Depth of Binary Tree

Given the root of a binary tree, return its maximum depth (# of nodes along longest root-to-leaf path).

Example

```
        3          depth 0
       / \
      9  20        depth 1
        / \
       15  7       depth 2
```

Output: 2

Both approaches:

Time: O(n)
Space: O(h) DFS / O(w) BFS

This is essentially the same recursion from the height() slide!

BFS Level-Count Alternative

```
def maxDepth(root):
    if not root:
       return -1
    depth, queue = -1, [root]
    while queue:
        n = len(queue)
        depth += 1
        for i in range(n):
        node = queue.popleft()
         if node.left:
              queue.append(node.left)
        if node.right:
              queue.append(node.right)
   return depth
```

Midterm Review

Array Lists • Linked Lists • Stacks • Queues • Big-O

What you should be able to do

- Pick the right data structure for a scenario
- State the key operations + typical running time
- Trace stack / queue behavior step-by-step
- Explain trade-offs: e.g. Arrays vs Linked Lists
- Compute Big-O for common code patterns

Array Lists (Dynamic Arrays)

Core idea: contiguous memory + indexable (Python list is a dynamic array).

Get by index: O(1).

Append to end: amortized O(1).

Insert / delete: O(n) (shift elements).

Great when you need fast access and you know the indexes of the elements you want; less great for frequent inserts/deletes.

Linked Lists

Core idea: nodes + pointers. Access is sequential.

Search / access the i-th element: O(n).

Insert / delete at head: O(1).

Singly vs doubly: doubly supports O(1) backward traversal + easier deletes, but uses extra space.

Great for frequent inserts/deletes near ends; not great for random access.

Stacks (LIFO)

Abstract operations: push(x), pop(), peek().

Typical implementations: dynamic array (list) or linked list (top at head).

With Python list: push() = append(), pop() = pop() from end, peek() = stack[-1]

Running time: push/pop are amortized O(1) with a dynamic array.

Common use: backtracking, parsing, undo, DFS, bracket matching.

Queues (FIFO)

Abstract operations: enqueue(x), dequeue(), first(), is_empty(), len(Q)

Naive array/list queue: enqueue at end is amortized O(1); dequeue from front is O(n) (shifts!).

Better array approach: circular buffer with head/tail indices ⇒ enqueue/dequeue amortized O(1).

Linked list queue: keep head + tail pointers ⇒ enqueue/dequeue O(1).

Common use: BFS, scheduling, buffering streams.

Big-O Quick Sheet

Big-O is an upper bound on growth as $n \to \infty$ (ignore constants & lower-order terms).

Common patterns: one loop ⇒ $O(n)$; nested loops ⇒ multiply; sequential blocks ⇒ add.

Typical runtimes: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$.

Amortized $O(1)$: average per operation over a long sequence (e.g., dynamic array append).

When in doubt: count how many times the *dominant* operation can happen.